

Evolución de la Educación Superior en Bolivia: Un Estudio Temporal sobre el Crecimiento y la Desigualdad de Género en Matrículas y Titulaciones

Dávalos Alarcón Rodrigo
Mamani Cahuana Daniel Edgar

Flores Ulo Wilmer Alejandro
Salinas Condori Ian Ezequiel

1 Introducción

El sistema de educación superior en Bolivia atraviesa un momento de transición crítica, donde la masificación de la matrícula no siempre se traduce en una reducción efectiva de las brechas sociales y económicas. En un entorno globalizado que demanda competencias técnicas de alto nivel, la universidad boliviana se enfrenta al desafío de ser un motor de movilidad social en un mercado laboral marcado por la informalidad. Sin embargo, para entender hacia dónde debe dirigirse la política educativa, es imperativo analizar primero de dónde venimos y cómo se comportan nuestros datos académicos actuales.

El presente estudio surge de la necesidad de transformar los registros históricos de matrículas y titulaciones en conocimiento estratégico. No basta con saber cuántos estudiantes ingresan al sistema; es fundamental identificar quiénes son, qué estudian, en qué departamentos se concentran y si el género sigue siendo un factor determinante en su trayectoria académica. Esta investigación propone un análisis descriptivo y prospectivo que permite visualizar las asimetrías del sistema universitario nacional, ofreciendo una base cuantitativa para la toma de decisiones.

Para garantizar la rigurosidad de este análisis, se ha adoptado la metodología CRISP-DM (Cross-Industry Standard Process for Data Mining), un estándar internacional que permite estructurar el ciclo de vida de los datos desde la comprensión del problema institucional hasta la generación de modelos de tendencia. A través de este enfoque, el informe desglosa la realidad educativa boliviana, evaluando la “macrocefalia” académica en el eje troncal y la persistencia de segregaciones que limitan el potencial del capital humano en el país.

A continuación, se detalla el marco contextual y la problemática que motivan este análisis, estableciendo los objetivos y la hoja de ruta metodológica que guiarán el desarrollo de la investigación.

2 Contextualización y Definición del Problema

2.1 Marco teórico y contextual

En el contexto global y particularmente en Bolivia, las universidades han asumido el reto de reducir la desigualdad, buscando democratizar el acceso al crecimiento profesional. Sin embargo, este incremento en la oferta educativa ha generado una paradoja: en lugar de equilibrar la balanza, ha consolidado la capacidad técnica de los centros donde se originaron estos modelos (como Europa), dejando rezagadas a otras regiones.

Bolivia, como nación latinoamericana, enfrenta esta brecha con un agravante estructural: el predominio del comercio informal sobre el empleo formal. Esta realidad subraya la importancia crítica de la educación superior; no solo se trata de masificar la matrícula, sino de garantizar que la formación académica permita al país competir en estándares internacionales a largo plazo.

El presente informe analiza los datos históricos de las universidades en Bolivia para diagnosticar el panorama educativo actual y evaluar si el camino trazado es el adecuado para los desafíos futuros. Si bien el mercado laboral presenta dificultades evidentes para los nuevos profesionales, este estudio no se centra en la empleabilidad, sino en el comportamiento de la accesibilidad a la educación visto desde perspectiva de las matrículas. Se parte de la premisa de que la educación superior mantiene su vigencia y utilidad, incluso en sectores tradicionalmente informales como los negocios. Por tanto, esta investigación se limita a un análisis descriptivo y prospectivo de los datos académicos disponibles.

2.2 Problemática

2.2.1 Problema principal

A pesar de los esfuerzos por masificar la educación superior en Bolivia, persiste una incertidumbre sobre la equidad y eficiencia del crecimiento de la matrícula. El problema central radica en la falta de un diagnóstico cuantitativo que determine si el aumento en el acceso a la universidad está reduciendo las brechas de género y desigualdades geográficas, o si simplemente está replicando modelos de concentración urbana y segregación por áreas de estudio que no se alinean con la realidad socioeconómica del país.

2.2.2 Problemas secundarios

- **Desconocimiento del ritmo de expansión:** No se tiene una medición precisa de la tasa de crecimiento anual de la matrícula, lo que impide determinar si el sistema universitario boliviano está en una fase de expansión sostenible o de estancamiento.
- **Segregación de género persistente:** Existe una brecha invisible en la elección de carreras y tasas de titulación entre hombres y mujeres, lo que sugiere que el acceso no es equitativo en todas las áreas del conocimiento (especialmente en sectores técnicos o de servicios).
- **Macrocefalia académica departamental:** La oferta y demanda educativa parecen estar altamente concentradas en el "eje troncal", dejando a los departamentos periféricos en una situación de desventaja competitiva y limitando la democratización del acceso profesional.
- **Desconexión entre formación y estructura económica:** Se observa una posible asimetría entre las áreas de estudio con mayor volumen de matriculados y las necesidades de un mercado laboral marcado por la informalidad, lo que podría generar una saturación en sectores de baja productividad.
- **Incertidumbre prospectiva:** La ausencia de proyecciones basadas en datos históricos dificulta la planificación estratégica de las instituciones educativas para anticipar la demanda futura de infraestructura y recursos docentes.

2.3 Objetivos

2.3.1 Objetivo General

Analizar la evolución temporal del crecimiento y las desigualdades de género en las matrículas y titulaciones de la educación superior en Bolivia, para identificar las áreas de estudio y ciudades con mayores brechas educativas.

2.3.2 Objetivos Específicos

- **Cuantificar la evolución de la matrícula:** Identificar las tasas de crecimiento anual de inscritos en el sistema universitario boliviano durante el periodo de estudio.
- **Identificar brechas de género:** Comparar la distribución de hombres y mujeres en las distintas áreas del conocimiento y niveles de titulación.
- **Analizar la concentración geográfica:** Determinar qué departamentos o ciudades concentran la mayor oferta y demanda académica, contrastándola con las zonas de menor acceso.
- **Evaluar la relación entre oferta y formalidad:** Explorar descriptivamente si las áreas de estudio con mayor crecimiento coinciden con los sectores económicos de mayor informalidad en el país.
- **Proyectar tendencias futuras:** Aplicar modelos descriptivos para estimar el comportamiento de la matrícula en los próximos años basándose en los datos históricos.

2.4 Justificación y alcance

2.5 Metodología usada

Para el análisis de los datos se usará lo que es la metodología CRISP-DM que es un acrónimo de de Cross-Industry Standard Process for Data Mining. Este es un modelo de proceso estándar, ampliamente reconocido y utilizado en la industria, que describe los pasos necesarios para llevar a cabo con éxito un proyecto de minería de datos o de análisis de datos. Este marco de trabajo proporciona una estructura iterativa y cíclica que organiza el flujo de trabajo en las siguientes seis fases:

1. **Comprensión del Negocio (Business Understanding):** Esta fase inicial se centra en la definición de los objetivos del proyecto y los requisitos desde una perspectiva de negocio o la institución.
2. **Comprensión de los Datos (Data Understanding):** Consiste en la recolección inicial de datos y la realización de actividades destinadas a familiarizarse con ellos.
3. **Preparación de los Datos (Data Preparation):** Esta etapa abarca todas las actividades necesarias para construir el conjunto de datos final que será introducido en las herramientas de modelado.
4. **Modelado (Modeling):** En esta fase se seleccionan y aplican diversas técnicas de modelado y se calibran sus parámetros a valores óptimos. Debido a que algunas técnicas requieren formatos de datos específicos, a menudo es necesario regresar a la fase de preparación de datos durante este proceso.
5. **Evaluación (Evaluation):** Una vez construido el modelo (o modelos), se debe evaluar su calidad, eficacia y si el modelo cumple con los objetivos definidos en la primera fase y se determina si existe alguna razón importante por la cual el modelo no deba ser utilizado.
6. **Despliegue (Deployment):** Los conocimientos obtenidos deben ser organizados y presentados de manera que el usuario final pueda utilizarlos.

La metodología CRISP-DM destaca por su naturaleza cíclica y su versatilidad, siendo un estándar agnóstico al sector o tecnología empleada. Su valor reside en su capacidad para alinear los requisitos técnicos con las metas estratégicas del negocio. La elección de este marco responde a su robustez y rigor, garantizando que el proceso siga un hilo conductor sistemático —desde la fase de observación hasta la evaluación final— sin omitir etapas fundamentales del método científico. Entre sus ventajas adicionales se encuentran: ::: {.content-visible when-format="revealjs"}
:::

6. **Despliegue (Deployment):** Los conocimientos obtenidos deben ser organizados y presentados de manera que el usuario final pueda utilizarlos

La metodología CRISP-DM destaca por su naturaleza cíclica y su versatilidad, siendo un estándar agnóstico al sector o tecnología empleada. Su valor reside en su capacidad para alinear los requisitos técnicos con las metas estratégicas del negocio. La elección de este marco responde a su robustez y rigor, garantizando que el proceso siga un hilo conductor sistemático -

- Replicabilidad y documentación
- Naturaleza iterativa y no lineal
- Mitigación de sesgos y error temprano (énfasis en la fase de comprensión de los datos)
- Alineación con el estado del arte (evaluación)

3 Procedimiento

3.1 Preparación

Importamos las librerías que usaremos:

```
import pandas as pd
import numpy as np
from scipy import stats
import matplotlib.pyplot as plt
import os
from sklearn.preprocessing import StandardScaler, MinMaxScaler
```

Cargamos los datos

```
df = pd.read_csv(os.path.join(DIR, "universities_bolivia_dataset.csv"))
```

Veremos algunos datos del dataset

```
print(f" Filas: {len(df):,} | Columnas: {df.shape[1]}")
print(f" Años: {df['AÑO'].min()} - {df['AÑO'].max()}")
print(f" Departamentos: {sorted(df['CIUDAD'].unique())}")
print(f" Tipo: {list(df['TIPO DE EDUCACIÓN'].unique())}")
```

Filas: 10,263 | Columnas: 11

Años: 2001 - 2016

Departamentos: ['BENI', 'CHUQUISACA', 'COCHABAMBA', 'LA PAZ', 'ORURO', 'PANDO', 'POTOSI', 'SANTA CRUZ', 'TARAPACA', 'TROPICAL', 'YACAJA']

Tipo: ['PUBLICA', 'PRIVADA']

4 Valores faltantes

Viendo el diagnóstico con NaN literales

```
nan_lit = df[COLS].isna().sum()
print("\n NaN literales por columna:")
for c in COLS:
    print(f" {CORTO[c]:<8} {nan_lit[c]:>6,}")
print(f" Total NaN literales: {nan_lit.sum()}")
```

NaN literales por columna:

Mat.H	0
Mat.M	0
Nuev.H	0
Nuev.M	0
Tit.H	0
Tit.M	0

Total NaN literales: 0

Haciendo un diagnóstico para ver todas las filas con un registro 0. Un registro con todos los valores en 0 no aporta información; tratarlos como NaN permite imputarlos correctamente.

```
todo_cero = df[COLS].sum(axis=1) == 0
print(f"\n Filas sin datos (todos ceros): {todo_cero.sum():,} "
      f"({todo_cero.mean()*100:.1f}% del total)")
print(f" Filas con al menos un valor : {(~todo_cero).sum():,}")

df_work = df.copy()
for c in COLS:
    df_work.loc[todo_cero, c] = np.nan
```

Filas sin datos (todos ceros): 2,301 (22.4% del total)

Filas con al menos un valor : 7,962

Viendo una tabla de resumen antes de imputar. La diferencia entre media y mediana revela distribuciones muy asimétricas ($\text{skew} > 3$ en todas las variables), lo que justifica usar la mediana como imputador en lugar de la media.

```
print(f"\n {'Variable':<12} {'Faltantes':>10} {'% total':>8} "
      f"{'Media':>9} {'Mediana':>9} {'Diferencia%':>12}")
print(f" {'-'*12} {'-'*10} {'-'*8} {'-'*9} {'-'*9} {'-'*12}")
for c in COLS:
    na = df_work[c].isna().sum()
    pct = na / len(df_work) * 100
    me = df_work[c].mean()
    md = df_work[c].median()
    dif = (me - md) / md * 100 if md != 0 else 0
    print(f" {CORTO[c]:<12} {na:>10,} {pct:>7.1f}% "
          f"{me:>9.1f} {md:>9.1f} {dif:>+11.1f}%")
```

Variable	Faltantes	% total	Media	Mediana	Diferencia%
Mat.H	2,301	22.4%	417.3	176.0	+137.1%
Mat.M	2,301	22.4%	408.0	141.0	+189.3%
Nuev.H	2,301	22.4%	176.7	76.0	+132.6%
Nuev.M	2,301	22.4%	175.1	64.0	+173.6%
Tit.H	2,301	22.4%	21.3	6.0	+255.6%
Tit.M	2,301	22.4%	25.2	6.0	+319.7%

Ahora se hará una imputación con mediana por grupo. Para ello se seguirán las estrategias de dos pasos:

- 1) Mediana del grupo ÁREA x TIPO DE EDUCACIÓN -> respeta el perfil típico de cada área y tipo de universidad.
- 2) Mediana global de la columna -> fallback para grupos que también quedaron sin datos.

```
for c in COLS:
    mediana_grupo = df_work.groupby(
        ["ÁREA", "TIPO DE EDUCACIÓN"])[c].transform("median")
    df_work[c] = df_work[c].fillna(mediana_grupo).fillna(df_work[c].median())

print(f"\n Método : mediana por grupo (ÁREA + TIPO DE EDUCACIÓN)")
print(f"  NaN restantes tras imputar: {df_work[COLS].isna().sum().sum()}")
```

Método : mediana por grupo (ÁREA + TIPO DE EDUCACIÓN)
NaN restantes tras imputar: 0

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle("2. Datos Faltantes", fontsize=15, fontweight="bold")

# Izquierda - barras de faltantes por variable
ax = axes[0]
colores_na = [CYAN, PINK, PURPLE, GREEN, ORANGE, YELLOW]
na_counts = [df[COLS].isna().sum()[c] + todo_cero.sum() for c in COLS]
# (NaN literales + filas todo-cero convertidas)
na_real = [df_work.isna().sum()[c] + 0 for c in COLS] # después de marcar
bars = ax.bar([CORTO[c] for c in COLS], [todo_cero.sum()] * len(COLS),
              color=colores_na, alpha=0.0) # placeholder altura
# recalcular con la copia marcada
na_vals = []
df_tmp = df.copy()
for c in COLS:
    df_tmp.loc[todo_cero, c] = np.nan
    na_vals.append(df_tmp[c].isna().sum())
ax.cla()
bars = ax.bar([CORTO[c] for c in COLS], na_vals,
              color=colores_na, edgecolor="white", linewidth=0.6)
for b, v in zip(bars, na_vals):
    ax.text(b.get_x() + b.get_width() / 2, b.get_height() + 8,
            f"{v:,}", ha="center", va="bottom", fontsize=9, fontweight="bold")
ax.set_title("Valores faltantes por variable\n(filas todo-cero → NaN)", fontsize=11)
ax.set_ylabel("Registros sin dato")
ax.grid(axis="y", alpha=0.25)

# Derecha - media vs mediana (justificación del imputador)
ax2 = axes[1]
x = np.arange(len(COLS))
medias = [df_work[c].mean() for c in COLS]
medianas = [df_work[c].median() for c in COLS]
ax2.bar(x - 0.18, medias, 0.35, label="Media", color=PINK, alpha=0.85)
ax2.bar(x + 0.18, medianas, 0.35, label="Mediana", color=CYAN, alpha=0.85)
```

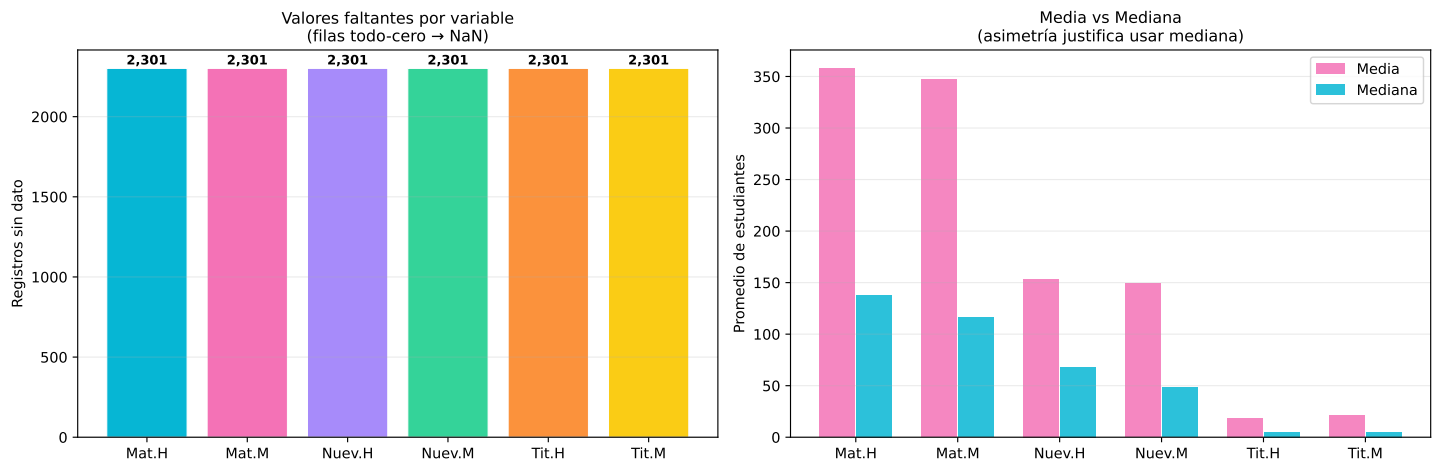
```

ax2.set_xticks(x)
ax2.set_xticklabels([CORTO[c] for c in COLS])
ax2.set_title("Media vs Mediana\n(asimetría justifica usar mediana)", fontsize=11)
ax2.set_ylabel("Promedio de estudiantes")
ax2.legend()
ax2.grid(axis="y", alpha=0.25)

plt.tight_layout()
# guardar(fig, "fig_datos_faltantes.png")
plt.show()

```

2. Datos Faltantes



4.1 Detección de outlier

La forma en la que identificaremos qué valores son atípicos será usando:

- Regla IQR ($Q3 + 1.5 \times IQR$).

Se tratarán usando dos estrategias:

- A) Eliminación — borrar las filas con outliers.
- B) Winsorización — reemplazar los extremos por los valores de los percentiles P5 y P95, conservando todas las filas. #

Comparar IQR con Winsorización como si fueran del mismo tipo es un error: el IQR es una regla de DETECCIÓN, mientras que eliminación y winsorización son estrategias de TRATAMIENTO. La comparación correcta es: Eliminación vs. Winsorización.

Se identifican como outliers los valores que superan $Q3 + 1.5 \times IQR$ (extremo superior). Solo se consideran registros con valor > 0 .

```

print("\n [DETECCIÓN] Regla IQR -  $Q3 + 1.5 \times IQR$ ")
print(f"  {'Var':<9} {'Q1':>7} {'Q3':>7} {'IQR':>7} "
      f"{'Lím.sup':>9} {'Outliers':>9} {'%':>6}")
print(f"  {'-'*9} {'-'*7} {'-'*7} {'-'*7} {'-'*9} {'-'*9} {'-'*6}")

iqr_res = {}
for c in COLS:

```

```

v = df_work[c][df_work[c] > 0]
q1, q3 = v.quantile(0.25), v.quantile(0.75)
iqr = q3 - q1
limite = q3 + 1.5 * iqr
n_out = (v > limite).sum()
iqr_res[c] = {"q1": q1, "q3": q3, "iqr": iqr, "limite": limite, "n": n_out}
print(f"  {CORTO[c]:<9} {q1:>7.0f} {q3:>7.0f} {iqr:>7.0f} "
      f"{limite:>9.0f} {n_out:>9,} {n_out/len(v)*100:>5.1f}%")

```

[DETECCIÓN]	Regla	IQR - Q3 + 1.5 × IQR				
Var	Q1	Q3	IQR	Lím.sup	Outliers	%
Mat.H	65	374	308	836	1,109	11.1%
Mat.M	37	395	358	932	1,010	10.1%
Nuev.H	36	170	134	371	970	10.1%
Nuev.M	22	179	157	414	847	8.9%
Tit.H	3	22	19	50	1,040	12.6%
Tit.M	3	27	24	63	932	11.7%

Se eliminan las filas donde AL MENOS UNA columna supera su límite IQR.

- Ventaja: elimina el ruido.
- Desventaja: se pierde información — reducción del dataset.

```

mask_outlier = pd.Series(False, index=df_work.index)
for c in COLS:
    mask_outlier |= (df_work[c] > iqr_res[c]["limite"])

df_elim = df_work[~mask_outlier].copy()
filas_elim = mask_outlier.sum()

print(f"\n  [TRATAMIENTO A] Eliminación de filas con outlier IQR")
print(f"  Filas eliminadas : {filas_elim:,} ({
    filas_elim/len(df_work)*100:.1f}%)")
print(f"  Filas restantes  : {len(df_elim):,}")
print(f"\n  {'Var':<9} {'Media orig':>11} {'→ Elim':>9} "
      f"{'Std orig':>10} {'→ Elim':>9} {'Red.Std%':>9}")
print(f"  {'-'*9} {'-'*11} {'-'*9} {'-'*10} {'-'*9} {'-'*9}")
elim_res = {}
for c in COLS:
    orig_m = df_work[c][df_work[c] > 0].mean()
    orig_s = df_work[c][df_work[c] > 0].std()
    new_m = df_elim[c][df_elim[c] > 0].mean()
    new_s = df_elim[c][df_elim[c] > 0].std()
    red = (orig_s - new_s) / orig_s * 100
    elim_res[c] = {"orig_m": orig_m, "new_m": new_m,
                  "orig_s": orig_s, "new_s": new_s, "red": red}
print(f"  {CORTO[c]:<9} {orig_m:>11.1f} {new_m:>9.1f} "
      f"{'-'*9} {orig_s:>10.1f} {new_s:>9.1f} {red:>8.1f}%")

```

[TRATAMIENTO A] Eliminación de filas con outlier IQR
 Filas eliminadas : 2,075 (20.2%)
 Filas restantes : 8,188

Var	Media orig	→ Elim	Std orig	→ Elim	Red.Std%
Mat.H	366.7	171.2	612.6	167.2	72.7%
Mat.M	357.8	157.8	639.8	179.8	71.9%
Nuev.H	164.2	78.8	266.0	68.2	74.4%
Nuev.M	160.6	73.4	291.4	77.4	73.4%
Tit.H	23.4	8.8	43.5	9.2	78.8%
Tit.M	27.3	10.7	48.1	12.7	73.5%

Los valores extremos se recortan al P5 (inferior) y al P95 (superior), reemplazándolos por esos percentiles. - Ventaja: no se pierde ninguna fila. - Desventaja: modifica los valores originales.

```
print(f"\n [TRATAMIENTO B] Winsorización P5-P95")
print(f"  {'Var':<9} {'Media orig':>11} {'→ Win':>9} "
      f"{'Std orig':>10} {'→ Win':>9} {'Afect.':>8} {'Red.Std%':>9}")
print(f"  {'-'*9} {'-'*11} {'-'*9} {'-'*10} {'-'*9} {'-'*8} {'-'*9}")

win_res = {}
for c in COLS:
    v = df_work[c][df_work[c] > 0]
    w = stats.mstats.winsorize(v, limits=[0.05, 0.05])
    n_af = int(((v < np.percentile(v, 5)) | (v > np.percentile(v, 95))).sum())
    red = (v.std() - float(w.std())) / v.std() * 100
    win_res[c] = {"orig_m": v.mean(), "win_m": float(w.mean()),
                  "orig_s": v.std(), "win_s": float(w.std()),
                  "n": n_af, "red": red}
    print(f"  {CORTO[c]:<9} {v.mean():>11.1f} {float(w.mean()):>9.1f} "
          f"{'v.std():>10.1f} {float(w.std()):>9.1f} {n_af:>8,} {red:>8.1f}%")
```

[TRATAMIENTO B] Winsorización P5-P95

Var	Media orig	→ Win	Std orig	→ Win	Afect.	Red.Std%
Mat.H	366.7	322.4	612.6	407.0	991	33.6%
Mat.M	357.8	304.8	639.8	409.4	961	36.0%
Nuev.H	164.2	140.2	266.0	161.0	952	39.5%
Nuev.M	160.6	133.5	291.4	168.5	839	42.2%
Tit.H	23.4	20.3	43.5	29.0	411	33.4%
Tit.M	27.3	23.7	48.1	33.2	400	30.8%

Ahora sí comparamos dos estrategias de TRATAMIENTO entre sí.

```
print(f"\n [COMPARATIVA] Eliminación vs. Winsorización")
print(f"  (ambas usan IQR como criterio de detección)")
print(f"  {'Var':<9} {'Red.Std Elim%':>14} {'Red.Std Win%':>13} "
      f"{'Filas perdidas':>15}")
print(f"  {'-'*9} {'-'*14} {'-'*13} {'-'*15}")
```

```

for c in COLS:
    print(f"  {CORTO[c]:<9} {elim_res[c]['red']:>13.1f}% "
          f"{win_res[c]['red']:>12.1f}% "
          f"{filas_elim:>12,} vs 0")
print(f"\n  → Eliminación reduce más la Std pero descarta "
      f"{filas_elim:,} filas ({filas_elim/len(df_work)*100:.1f}%).")

```

[COMPARATIVA] Eliminación vs. Winsorización
(ambas usan IQR como criterio de detección)

Var	Red.Std Elim%	Red.Std Win%	Filas perdidas
Mat.H	72.7%	33.6%	2,075 vs 0
Mat.M	71.9%	36.0%	2,075 vs 0
Nuev.H	74.4%	39.5%	2,075 vs 0
Nuev.M	73.4%	42.2%	2,075 vs 0
Tit.H	78.8%	33.4%	2,075 vs 0
Tit.M	73.5%	30.8%	2,075 vs 0

→ Eliminación reduce más la Std pero descarta 2,075 filas (20.2%).

Winsorización conserva todas las filas a cambio de modificar los valores extremos. En este dataset se prefiere Winsorización para no perder información de carreras con matrículas altas.

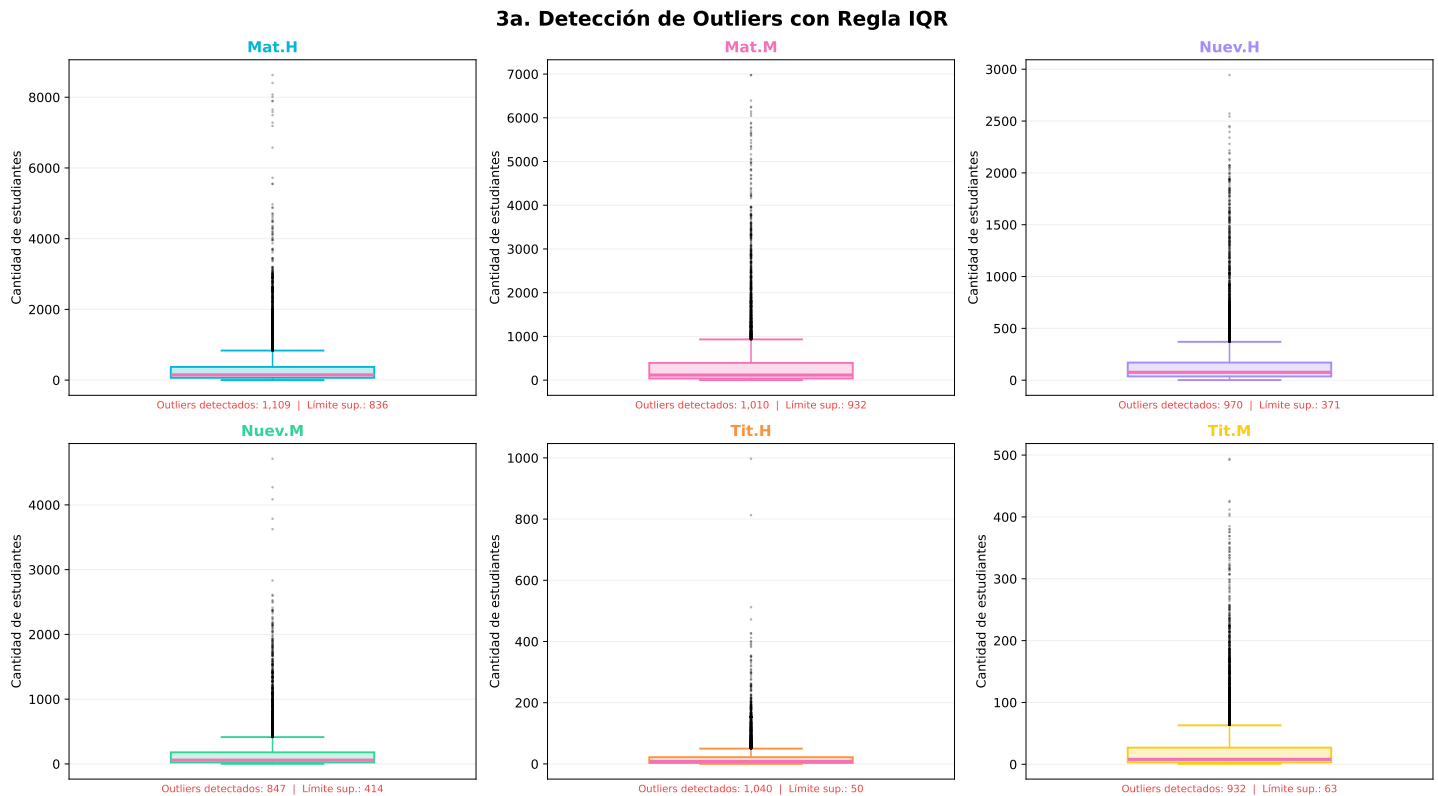
Viendo una gráfica

```

fig, axes = plt.subplots(2, 3, figsize=(16, 9))
fig.suptitle("3a. Detección de Outliers con Regla IQR",
             fontsize=16, fontweight="bold")
colores_box = [CYAN, PINK, PURPLE, GREEN, ORANGE, YELLOW]
for i, c in enumerate(COLS):
    ax = axes[i // 3][i % 3]
    data = df_work[c][df_work[c] > 0]
    ax.boxplot(
        data, patch_artist=True, widths=0.5,
        boxprops=dict(facecolor=colores_box[i] + "40",
                      edgecolor=colores_box[i], linewidth=1.5),
        medianprops=dict(color=PINK, linewidth=2.5),
        whiskerprops=dict(color=colores_box[i], linewidth=1.2),
        capprops=dict(color=colores_box[i], linewidth=1.5),
        flierprops=dict(marker=".", color=RED, markersize=2, alpha=0.3),
    )
    ax.set_title(CORTO[c], color=colores_box[i], fontweight="bold")
    ax.set_xlabel(
        f"Outliers detectados: {iqr_res[c]['n']:,} | "
        f"Límite sup.: {iqr_res[c]['limite']:.0f}",
        fontsize=8, color=RED,
    )
    ax.set_xticks([])
    ax.set_ylabel("Cantidad de estudiantes")
    ax.grid(alpha=0.2)
plt.tight_layout()

```

```
plt.show()
guardar(fig, "fig_outliers_deteccion_iqr.png")
```



fig_outliers_deteccion_iqr.png

Segunda visualización

```
fig, axes = plt.subplots(1, 3, figsize=(18, 6))
fig.suptitle(
    "3b. Tratamiento de Outliers - Eliminación vs. Winsorización\n"
    "(detección previa con regla IQR en ambos casos)",
    fontsize=13, fontweight="bold",
)

labels = [CORTO[c] for c in COLS]
x = np.arange(len(labels))

# Panel izquierdo - reducción de Std
ax = axes[0]
ax.bar(x - 0.18, [elim_res[c]["red"] for c in COLS], 0.35,
      label="Eliminación", color=RED, alpha=0.85)
ax.bar(x + 0.18, [win_res[c]["red"] for c in COLS], 0.35,
      label="Winsorización", color=GREEN, alpha=0.85)
ax.set_xticks(x)
ax.set_xticklabels(labels, rotation=15)
ax.set_title("Reducción de Desv. Estándar (%)", fontsize=11)
ax.set_ylabel("Reducción (%)")
ax.legend()
ax.grid(axis="y", alpha=0.2)
```

```

# Panel central - media antes/después por tratamiento
ax2 = axes[1]
medias_orig = [df_work[c][df_work[c] > 0].mean() for c in COLS]
medias_elim = [elim_res[c]["new_m"] for c in COLS]
medias_win = [win_res[c]["win_m"] for c in COLS]
ax2.bar(x - 0.25, medias_orig, 0.24,
        label="Original", color=BLUE, alpha=0.85)
ax2.bar(x, medias_elim, 0.24,
        label="Eliminación", color=RED, alpha=0.85)
ax2.bar(x + 0.25, medias_win, 0.24,
        label="Winsorización", color=GREEN, alpha=0.85)
ax2.set_xticks(x)
ax2.set_xticklabels(labels, rotation=15)
ax2.set_title("Media original vs. tras tratamiento", fontsize=11)
ax2.set_ylabel("Media de estudiantes")
ax2.legend()
ax2.grid(axis="y", alpha=0.2)

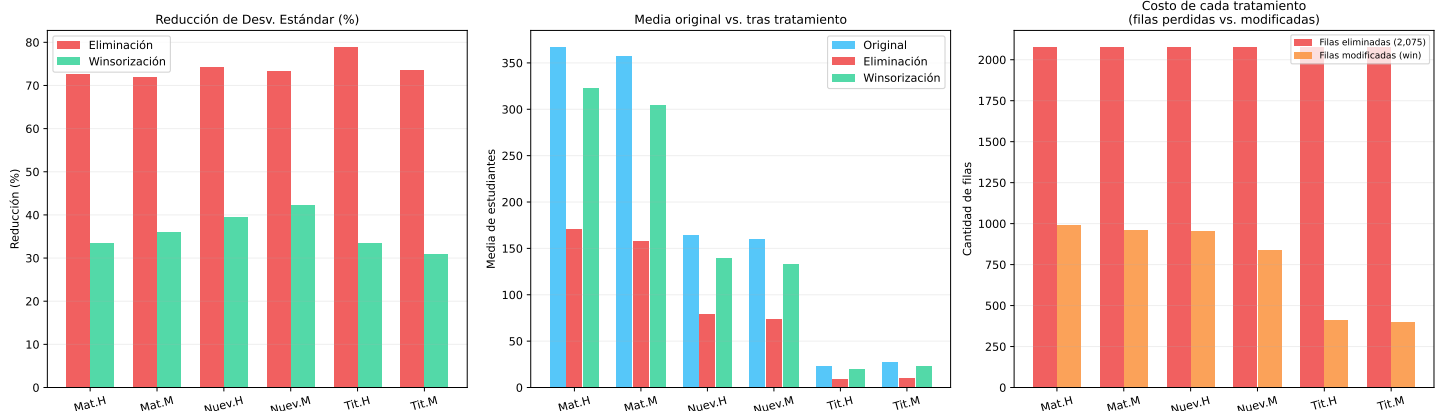
# Panel derecho - costo: filas perdidas vs. filas modificadas
ax3 = axes[2]
filas_mod_win = [win_res[c]["n"] for c in COLS]
ax3.bar(x - 0.18, [filas_elim] * len(COLS), 0.35,
        label=f"Filas eliminadas ({filas_elim:})", color=RED, alpha=0.85)
ax3.bar(x + 0.18, filas_mod_win, 0.35,
        label="Filas modificadas (win)", color=ORANGE, alpha=0.85)
ax3.set_xticks(x)
ax3.set_xticklabels(labels, rotation=15)
ax3.set_title(
    "Costo de cada tratamiento\n(filas perdidas vs. modificadas)", fontsize=11)
ax3.set_ylabel("Cantidad de filas")
ax3.legend(fontsize=8)
ax3.grid(axis="y", alpha=0.2)

plt.tight_layout()
plt.show()

guardar(fig, "fig_outliers_tratamiento_comparativa.png")

```

3b. Tratamiento de Outliers — Eliminación vs. Winsorización
(detección previa con regla IQR en ambos casos)



5 Metricas descriptivas

```

resumen = []
for c in COLS:
    v = df_work[c]
    resumen.append({
        "Var": CORTO[c], "N": len(v), "Media": v.mean(),
        "Mediana": v.median(), "Std": v.std(),
        "Mín": v.min(), "Máx": v.max(),
        "Asimetría": v.skew(), "Curtosis": v.kurtosis(),
    })
print(pd.DataFrame(resumen).to_string(
    index=False, float_format="{:.1f}".format))

print("\n    Por Tipo de Educación  ")
print(df_work.groupby("TIPO DE EDUCACIÓN")[
    COLS].mean().round(1).rename(columns=CORTO))

print("\n    Por Departamento (Matriculados)  ")
cm = df_work.groupby("CIUDAD")[
    ["MATRICULADOS HOMBRES", "MATRICULADOS MUJERES"]].mean().round(1)
cm.columns = ["Hombres", "Mujeres"]
cm["Total"] = cm.sum(axis=1)
print(cm.sort_values("Total", ascending=False))

```

Var	N	Media	Mediana	Std	Mín	Máx	Asimetría	Curtosis
Mat.H	10263	357.8	138.0	607.8	0.0	8628.0	4.7	35.5
Mat.M	10263	347.0	116.0	633.0	0.0	6978.0	4.1	23.1
Nuev.H	10263	153.5	68.0	260.3	0.0	2944.0	4.1	22.0
Nuev.M	10263	149.2	48.0	283.8	0.0	4712.0	4.9	36.7
Tit.H	10263	18.9	5.0	40.1	0.0	997.0	5.8	70.2
Tit.M	10263	21.3	5.0	43.9	0.0	494.0	4.0	21.2

Por Tipo de Educación

	Mat.H	Mat.M	Nuev.H	Nuev.M	Tit.H	Tit.M
TIPO DE EDUCACIÓN						
PRIVADA	200.7	183.1	129.6	108.3	14.4	13.8
PUBLICA	522.1	518.3	178.4	191.9	23.5	29.1

Por Departamento (Matriculados)

	Hombres	Mujeres	Total
CIUDAD			
LA PAZ	490.7	456.0	946.7
SANTA CRUZ	440.9	450.8	891.7
COCHABAMBA	376.5	356.1	732.6
ORURO	307.6	256.8	564.4
CHUQUISACA	271.4	269.2	540.6
POTOSI	270.0	265.1	535.1

TARIJA	233.0	232.6	465.6
BENI	213.3	220.4	433.7
PANDO	128.9	173.7	302.6

- Normalización: Lleva los valores a 0 y 1. Se recomienda para redes neuronales.
- Estandarización: Lleva los valores con media 0 y desviación estándar

de 1. Se recomienda para SVM, árboles de decisión, PCA. Esta no se aplica al tiempo por que se pierde la relación temporal

Aplicaremos ambos casos por si acaso para en un futuro usar en diferentes casos. Usaremos Scikit-Learn, ya que nos permitirá poder realizar el train spit posterior.

```
COLS_TO_STANDARDIZE = COLS.copy()
print("Columnas a estandarizar:", COLS_TO_STANDARDIZE)
scaler_standard_scaler = StandardScaler()
df_work_standard = df_work.copy()
df_work_standard[COLS_TO_STANDARDIZE] = scaler_standard_scaler.fit_transform(
    df_work_standard[COLS])

print("\nMuestra:\n")
print(df_work_standard[COLS_TO_STANDARDIZE].sample(n=5))
print("\nDescripción:\n")
print(df_work_standard[COLS_TO_STANDARDIZE].describe())
```

Columnas a estandarizar: ['MATRICULADOS HOMBRES', 'MATRICULADOS MUJERES', 'NUEVOS HOMBRES', 'NUEVOS MUJERES']

Muestra:

	MATRICULADOS HOMBRES	MATRICULADOS MUJERES	NUEVOS HOMBRES \
9284	-0.045767	-0.058388	-0.420456
2235	-0.389643	3.466418	-0.320578
1359	-0.485073	-0.450209	-0.347468
9299	-0.457102	-0.429670	-0.397407
3917	-0.588730	-0.548164	-0.589482

	NUEVOS MUJERES	TITULADOS HOMBRES	TITULADOS MUJERES
9284	-0.405789	-0.295639	-0.347550
2235	2.088926	-0.395327	1.451171
1359	-0.307128	-0.470092	-0.484162
9299	-0.300081	-0.420249	-0.370319
3917	-0.525592	-0.345483	-0.438625

Descripción:

	MATRICULADOS HOMBRES	MATRICULADOS MUJERES	NUEVOS HOMBRES \
count	1.026300e+04	1.026300e+04	1.026300e+04
mean	-2.215470e-17	-2.215470e-17	-3.323205e-17
std	1.000049e+00	1.000049e+00	1.000049e+00
min	-5.887297e-01	-5.481642e-01	-5.894817e-01
25%	-4.949453e-01	-4.991867e-01	-4.742371e-01
50%	-3.616727e-01	-3.648933e-01	-3.282607e-01
75%	2.580498e-02	7.590500e-02	1.747310e-02

max	1.360727e+01	1.047653e+01	1.071985e+01
-----	--------------	--------------	--------------

	NUEVOS MUJERES	TITULADOS HOMBRES	TITULADOS MUJERES
count	10263.000000	1.026300e+04	1.026300e+04
mean	0.000000	-5.538675e-18	-3.323205e-17
std	1.000049	1.000049e+00	1.000049e+00
min	-0.525592	-4.700922e-01	-4.841621e-01
25%	-0.469214	-4.451703e-01	-4.613935e-01
50%	-0.356459	-3.454830e-01	-3.703190e-01
75%	0.045233	-7.134279e-02	-2.878967e-02
max	16.077653	2.437698e+01	1.076354e+01

```
COLS_TO_NORMALIZE = COLS.copy()
print("Columnas a estandarizar:", COLS_TO_NORMALIZE)
scaler_min_max_scaler = MinMaxScaler()
df_work_min_max_scaler = df_work.copy()
df_work_min_max_scaler[COLS_TO_NORMALIZE] = scaler_min_max_scaler.fit_transform(
    df_work_min_max_scaler[COLS])

print("\nMuestra:\n")
print(df_work_min_max_scaler[COLS_TO_NORMALIZE].sample(n=5))
print("\nDescripción:\n")
print(df_work_min_max_scaler[COLS_TO_NORMALIZE].describe())
```

Columnas a estandarizar: ['MATRICULADOS HOMBRES', 'MATRICULADOS MUJERES', 'NUEVOS HOMBRES', 'NUEVOS MUJERES']

Muestra:

	MATRICULADOS HOMBRES	MATRICULADOS MUJERES	NUEVOS HOMBRES	\
6004	0.010315	0.004299	0.015625	
2027	0.000000	0.000000	0.000000	
3303	0.000000	0.000000	0.000000	
9587	0.019703	0.037690	0.043478	
9291	0.000927	0.005016	0.004076	

	NUEVOS MUJERES	TITULADOS HOMBRES	TITULADOS MUJERES
6004	0.003396	0.002006	0.002024
2027	0.003820	0.000000	0.000000
3303	0.000000	0.004012	0.004049
9587	0.030560	0.001003	0.008097
9291	0.004669	0.000000	0.004049

Descripción:

	MATRICULADOS HOMBRES	MATRICULADOS MUJERES	NUEVOS HOMBRES	\
count	10263.000000	10263.000000	10263.000000	
mean	0.041472	0.049721	0.052123	
std	0.070446	0.090710	0.088427	
min	0.000000	0.000000	0.000000	
25%	0.006606	0.004443	0.010190	
50%	0.015994	0.016624	0.023098	
75%	0.043289	0.056606	0.053668	

	max	1.000000	1.000000	1.000000
		NUEVOS MUJERES	TITULADOS HOMBRES	TITULADOS MUJERES
count	10263.000000	10263.000000	10263.000000	
mean	0.031656	0.018919	0.043045	
std	0.060232	0.040248	0.088911	
min	0.000000	0.000000	0.000000	
25%	0.003396	0.001003	0.002024	
50%	0.010187	0.005015	0.010121	
75%	0.034380	0.016048	0.040486	
max	1.000000	1.000000	1.000000	

En función de los modelos escogidos posteriormente se podrá decidir por una de las dos transformaciones.

Para ello vemos que las columnas que se pueden cambiar de tipo son: 'ÁREA', 'CARRERA', 'CIUDAD', 'AÑO', 'TIPO DE EDUCACIÓN'

Viendo de que tipo son las columnas:

Creando copia de seguridad

```
df_change_type = df_work.copy()
```

```
CATEGORICAL_COLS = ["ÁREA", "CARRERA", "CIUDAD", "TIPO DE EDUCACIÓN"]
print(f"Convirtiendo {CATEGORICAL_COLS} a categórica")
df_change_type[CATEGORICAL_COLS] = df_change_type[CATEGORICAL_COLS].astype(
    "category")
```

Convirtiendo ['ÁREA', 'CARRERA', 'CIUDAD', 'TIPO DE EDUCACIÓN'] a categórica

No es posible convertir el año a pd.date_time por que no se tiene ni meses ni días. Los caminos que se recomiendan depende del enfoque que le daremos: - Serie de tiempo: Se puede estandarizar - Sin enfoque temporal: Se puede tratar como una variable categórica

```
df_change_type["AÑO"] = df_change_type["AÑO"].astype("category")
```

Viendo de que tipo son las columnas:

```
print(df_change_type[["ÁREA", "CARRERA", "CIUDAD",
    "AÑO", "TIPO DE EDUCACIÓN"]].dtypes)
```

```
ÁREA          category
CARRERA       category
CIUDAD        category
AÑO           category
TIPO DE EDUCACIÓN category
dtype: object
```

```

fig, axes = plt.subplots(2, 3, figsize=(16, 9))
fig.suptitle("Diagramas de Caja", fontsize=16, fontweight="bold")
for i, c in enumerate(COLS):
    ax = axes[i // 3][i % 3]
    data = df_work[c][df_work[c] > 0]
    bp = ax.boxplot(data, patch_artist=True, widths=0.5,
                    boxprops=dict(
                        facecolor=colores_box[i] + "40", edgecolor=colores_box[i]),
                    medianprops=dict(color=PINK, linewidth=2),
                    flierprops=dict(marker=".", color=RED, markersize=2, alpha=0.3))
    ax.set_title(CORTO[c], color=colores_box[i], fontweight="bold")
    ax.set_xlabel(f"Outliers IQR: {iqr_res[c]['n']:,}", fontsize=8, color=RED)
    ax.grid(alpha=0.2)
plt.tight_layout()
plt.show()
guardar(fig, "fig1_boxplots.png")

```

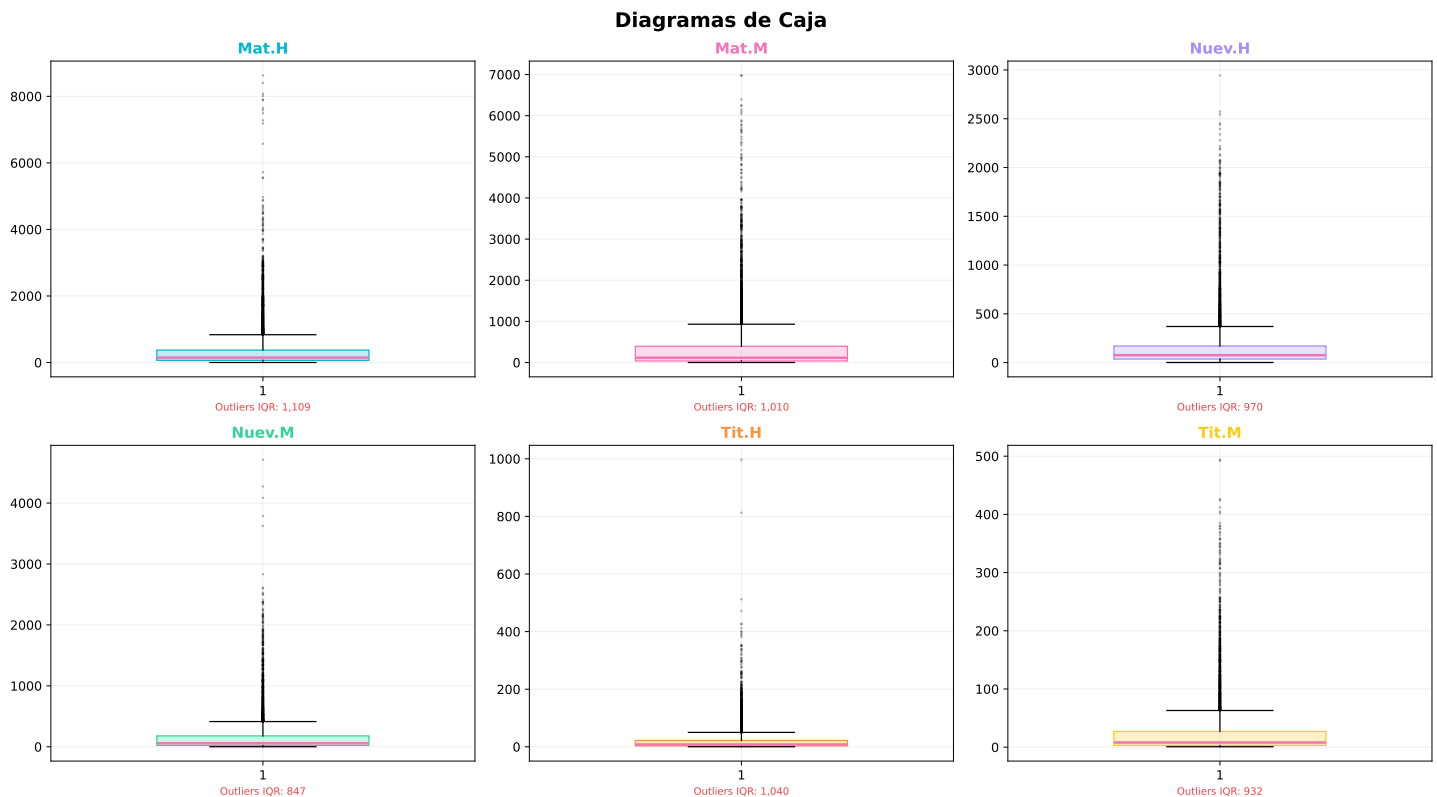


fig1_boxplots.png

```

fig, axes = plt.subplots(1, 3, figsize=(18, 6))
fig.suptitle(
    "3b. Tratamiento de Outliers - Eliminación vs. Winsorización\n"
    "(detección previa con regla IQR en ambos casos)",
    fontsize=13, fontweight="bold",
)
labels = [CORTO[c] for c in COLS]
x = np.arange(len(labels))

```

```

# Panel izquierdo - reducción de Std
ax = axes[0]
ax.bar(x - 0.18, [elim_res[c]["red"] for c in COLS], 0.35,
      label="Eliminación", color=RED, alpha=0.85)
ax.bar(x + 0.18, [win_res[c]["red"] for c in COLS], 0.35,
      label="Winsorización", color=GREEN, alpha=0.85)
ax.set_xticks(x)
ax.set_xticklabels(labels, rotation=15)
ax.set_title("Reducción de Desv. Estándar (%)", fontsize=11)
ax.set_ylabel("Reducción (%)")
ax.legend()
ax.grid(axis="y", alpha=0.2)

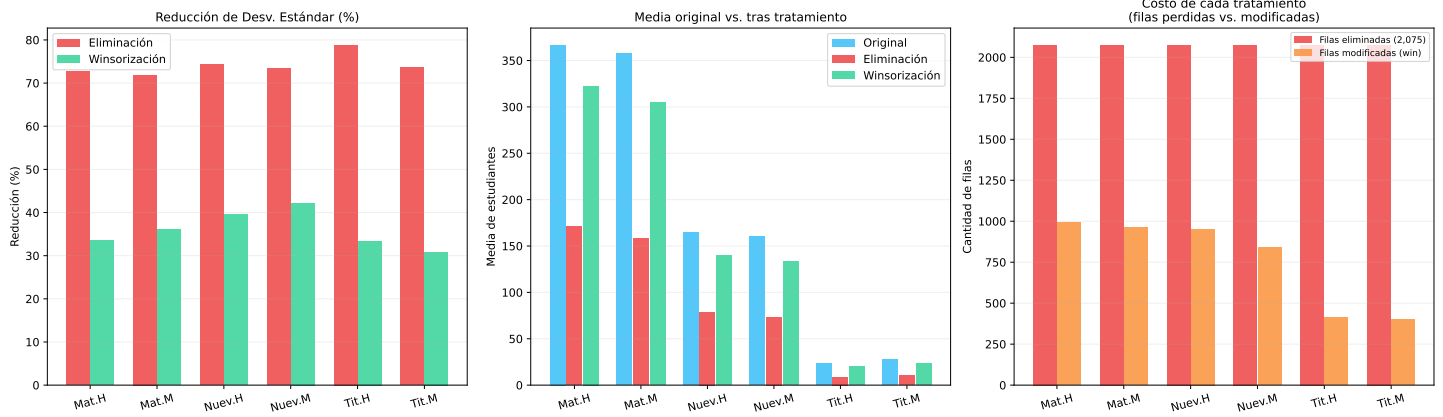
# Panel central - media antes/después por tratamiento
ax2 = axes[1]
medias_orig = [df_work[c][df_work[c] > 0].mean() for c in COLS]
medias_elim = [elim_res[c]["new_m"] for c in COLS]
medias_win = [win_res[c]["win_m"] for c in COLS]
ax2.bar(x - 0.25, medias_orig, 0.24,
      label="Original", color=BLUE, alpha=0.85)
ax2.bar(x, medias_elim, 0.24,
      label="Eliminación", color=RED, alpha=0.85)
ax2.bar(x + 0.25, medias_win, 0.24,
      label="Winsorización", color=GREEN, alpha=0.85)
ax2.set_xticks(x)
ax2.set_xticklabels(labels, rotation=15)
ax2.set_title("Media original vs. tras tratamiento", fontsize=11)
ax2.set_ylabel("Media de estudiantes")
ax2.legend()
ax2.grid(axis="y", alpha=0.2)

# Panel derecho - costo: filas perdidas vs. filas modificadas
ax3 = axes[2]
filas_mod_win = [win_res[c]["n"] for c in COLS]
ax3.bar(x - 0.18, [filas_elim] * len(COLS), 0.35,
      label=f"Filas eliminadas ({filas_elim:})", color=RED, alpha=0.85)
ax3.bar(x + 0.18, filas_mod_win, 0.35,
      label="Filas modificadas (win)", color=ORANGE, alpha=0.85)
ax3.set_xticks(x)
ax3.set_xticklabels(labels, rotation=15)
ax3.set_title(
    "Costo de cada tratamiento\n(filas perdidas vs. modificadas)", fontsize=11)
ax3.set_ylabel("Cantidad de filas")
ax3.legend(fontsize=8)
ax3.grid(axis="y", alpha=0.2)

plt.tight_layout()
plt.show()
guardar(fig, "fig_outliers_tratamiento_comparativa.png")

```

3b. Tratamiento de Outliers — Eliminación vs. Winsorización (detección previa con regla IQR en ambos casos)



fig_outliers_tratamiento_comparativa.png

```
fig, ax = plt.subplots(figsize=(12, 7))
fig.suptitle("Matriculados Promedio por Área", fontsize=14, fontweight="bold")
am = df_work.groupby(
    "ÁREA")["MATRICULADOS HOMBRES", "MATRICULADOS MUJERES"].mean()
am["short"] = am.index.map(AREA_CORTO)
am = am.sort_values("MATRICULADOS HOMBRES")
y = range(len(am))
ax.barh([i - 0.18 for i in y], am["MATRICULADOS HOMBRES"],
        0.35, label="Hombres", color=CYAN)
ax.barh([i + 0.18 for i in y], am["MATRICULADOS MUJERES"],
        0.35, label="Mujeres", color=PINK)
ax.set_yticks(list(y))
ax.set_yticklabels(am["short"])
ax.legend()
ax.grid(axis="x", alpha=0.2)
plt.tight_layout()
plt.show()
guardar(fig, "fig3_barras_area.png")
```

Matriculados Promedio por Área

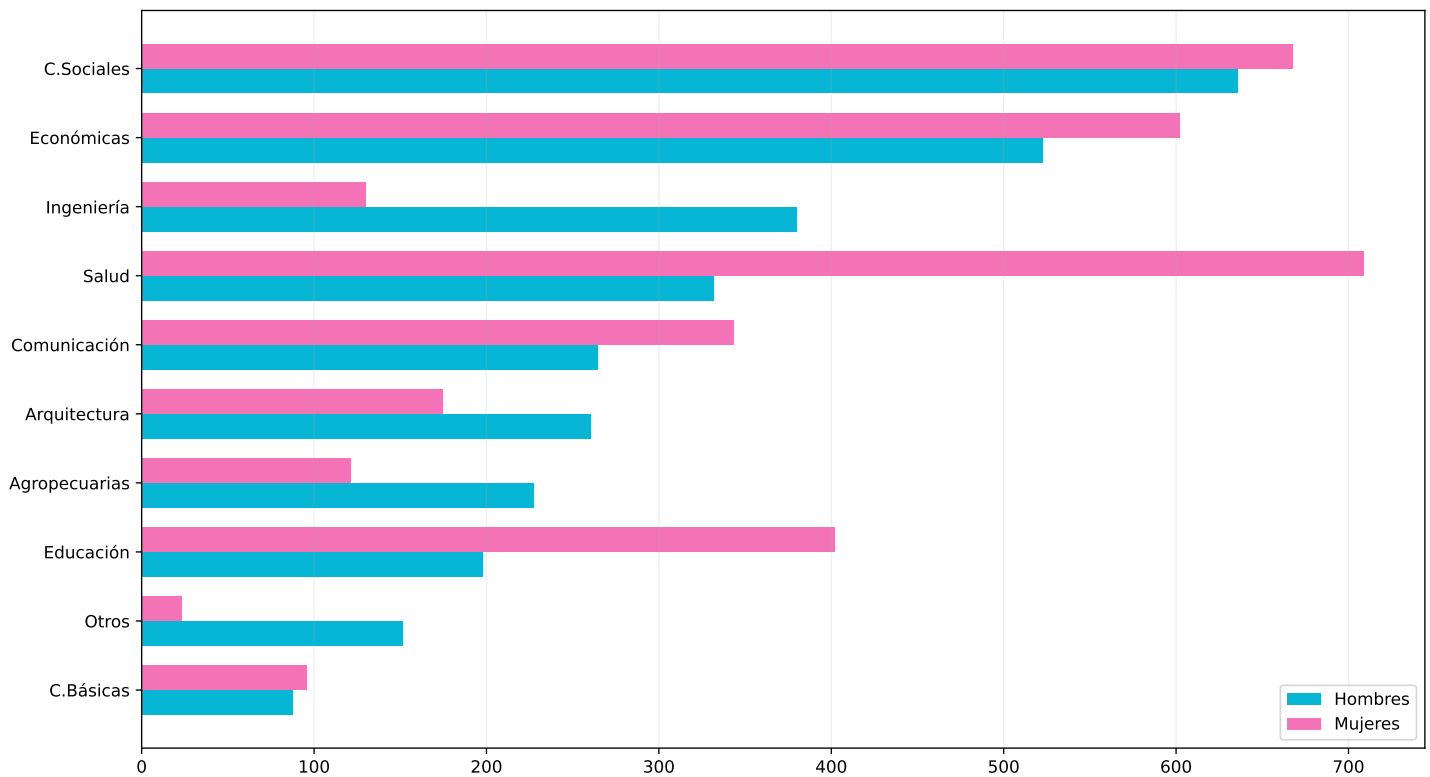


fig3_barras_area.png

```
fig, ax = plt.subplots(figsize=(12, 6))
fig.suptitle("Matriculados Promedio por Departamento",
             fontsize=14, fontweight="bold")
cm2 = df_work.groupby("CIUDAD")[
    ["MATRICULADOS HOMBRES", "MATRICULADOS MUJERES"]].mean()
cm2 = cm2.sort_values("MATRICULADOS HOMBRES")
y = range(len(cm2))
ax.barh([i - 0.18 for i in y], cm2["MATRICULADOS HOMBRES"],
        0.35, label="Hombres", color=CYAN)
ax.barh([i + 0.18 for i in y], cm2["MATRICULADOS MUJERES"],
        0.35, label="Mujeres", color=PINK)
ax.set_yticks(list(y))
ax.set_yticklabels(cm2.index)
ax.legend()
ax.grid(axis="x", alpha=0.2)
plt.tight_layout()
plt.show()
guardar(fig, "fig4_barras_ciudad.png")
```

Matriculados Promedio por Departamento

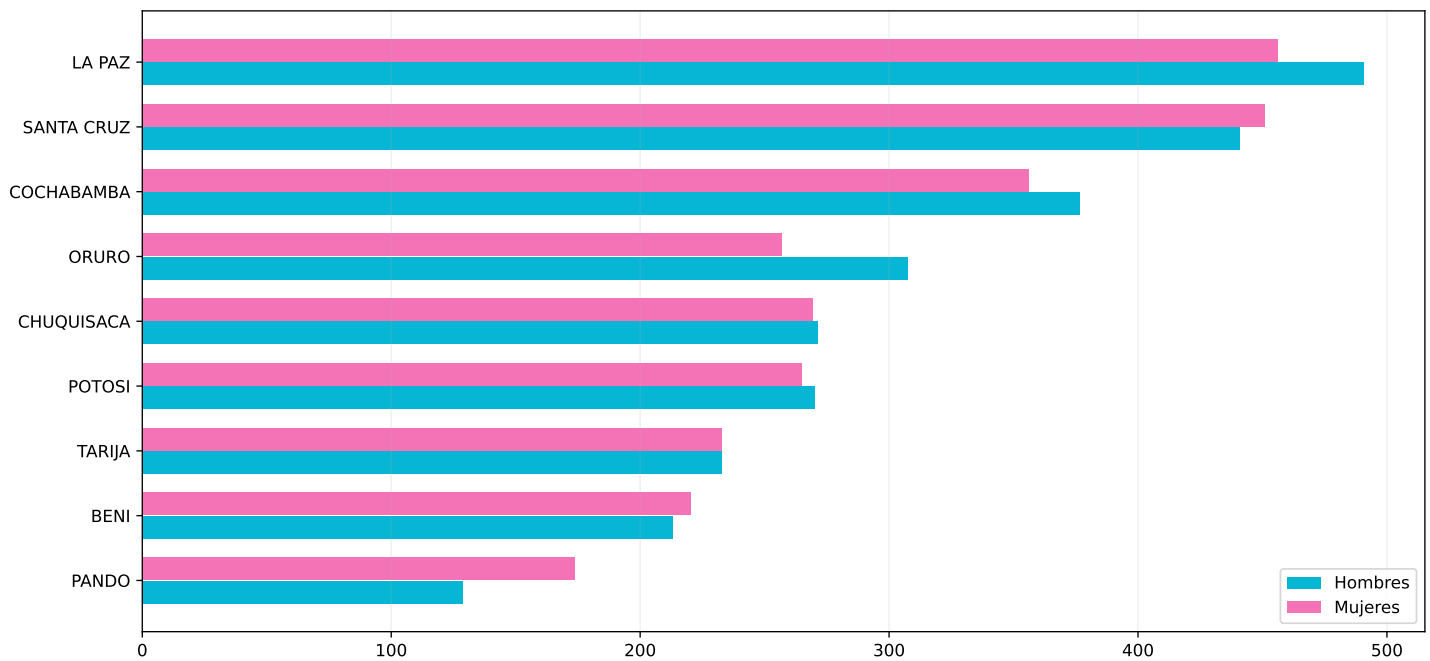


fig4_barras_ciudad.png

```
fig, ax = plt.subplots(figsize=(12, 6))
fig.suptitle("Evolución de Matrícula Total por Año",
             fontsize=14, fontweight="bold")
yt = df_work.groupby("AÑO")[["MATRICULADOS HOMBRES",
                             "MATRICULADOS MUJERES"]].sum() / 1000
ax.plot(yt.index, yt["MATRICULADOS HOMBRES"], "-o",
        color=CYAN, lw=2.5, ms=6, label="Hombres")
ax.plot(yt.index, yt["MATRICULADOS MUJERES"], "-o",
        color=PINK, lw=2.5, ms=6, label="Mujeres")
ax.fill_between(yt.index, yt["MATRICULADOS HOMBRES"], yt["MATRICULADOS MUJERES"],
               alpha=0.15, color=PURPLE)
ax.axvline(2012, ls="--", color=YELLOW, alpha=0.5)
ax.annotate("2012: cruce de género", xy=(2012, yt.iloc[6].mean()), fontsize=9,
           color=YELLOW, ha="center")
ax.set_xlabel("Año")
ax.set_ylabel("Matriculados (miles)")
ax.legend()
ax.grid(alpha=0.2)
plt.tight_layout()
guardar(fig, "fig5_evolucion.png")
```

fig5_evolucion.png

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))
fig.suptitle("Comparativa: IQR vs Winsorización",
             fontsize=14, fontweight="bold")
labels = [CORTO[c] for c in COLS]
x = np.arange(len(labels))
```

```

ax1.bar(x - 0.18, [iqr_res[c]["n"]
               for c in COLS], 0.35, label="IQR", color=RED)
ax1.bar(x + 0.18, [win_res[c]["n"]
               for c in COLS], 0.35, label="Winsor", color=ORANGE)
ax1.set_xticks(x)
ax1.set_xticklabels(labels, rotation=30)
ax1.set_title("Cantidad de outliers detectados", fontsize=11)
ax1.legend()
ax1.grid(axis="y", alpha=0.2)

ax2.bar(x - 0.18, [win_res[c]["orig_s"]
               for c in COLS], 0.35, label="Std Original", color=PURPLE)
ax2.bar(x + 0.18, [float(win_res[c]["win_s"])
               for c in COLS], 0.35, label="Std Winsorizada", color=GREEN)
ax2.set_xticks(x)
ax2.set_xticklabels(labels, rotation=30)
ax2.set_title("Reducción de Desv. Estándar", fontsize=11)
ax2.legend()
ax2.grid(axis="y", alpha=0.2)

plt.tight_layout()
guardar(fig, "fig6_comparativa_outliers.png")

```

fig6_comparativa_outliers.png

```

fig, ax = plt.subplots(figsize=(10, 5))
fig.suptitle("Media vs Mediana (justifica usar mediana para imputar)",
             fontsize=13, fontweight="bold")
medias = [df_work[c].mean() for c in COLS]
medianas = [df_work[c].median() for c in COLS]
ax.bar(x - 0.18, medias, 0.35, label="Media", color=PINK)
ax.bar(x + 0.18, medianas, 0.35, label="Mediana", color=CYAN)
ax.set_xticks(x)
ax.set_xticklabels(labels)
ax.legend()
ax.grid(axis="y", alpha=0.2)
plt.tight_layout()
plt.show()
guardar(fig, "fig7_media_vs_mediana.png")

```

Media vs Mediana (justifica usar mediana para imputar)

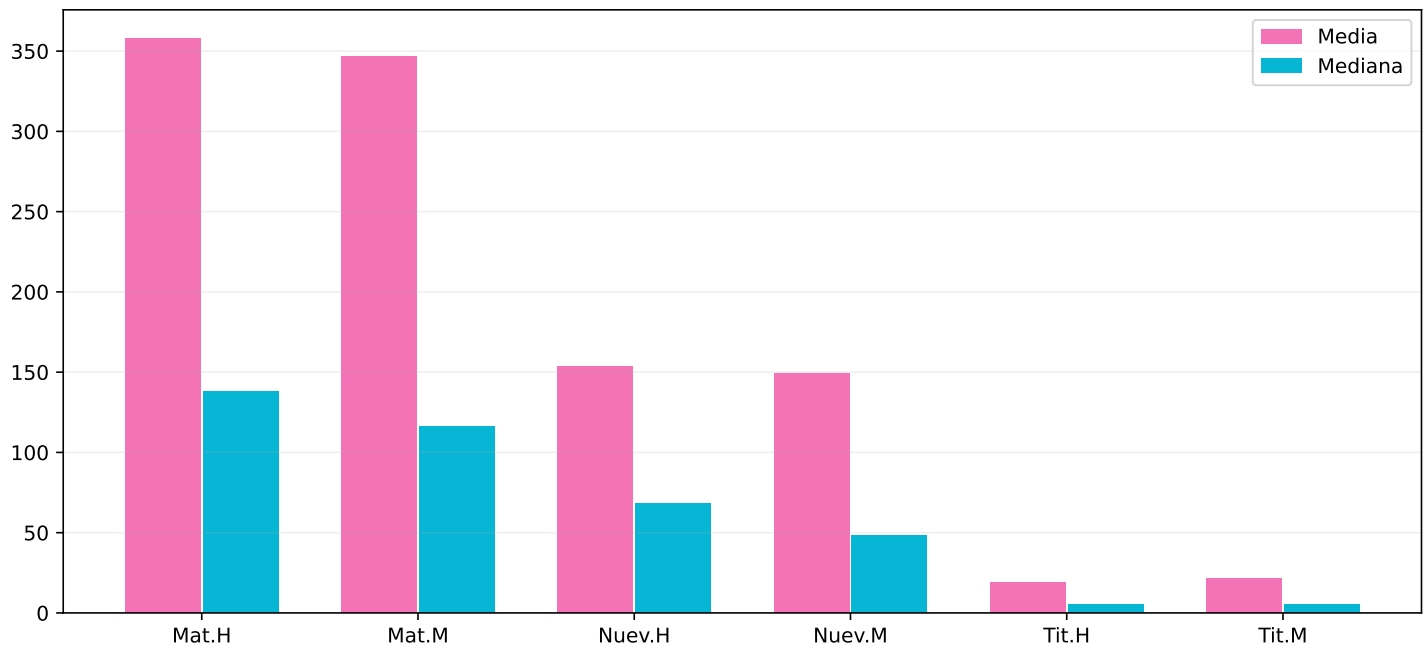


fig7_media_vs_mediana.png

```
fig, ax = plt.subplots(figsize=(10, 5))
fig.suptitle("Media vs Mediana (justifica usar mediana para imputar)",
             fontsize=13, fontweight="bold")
medias = [df_work[c].mean() for c in COLS]
medianas = [df_work[c].median() for c in COLS]
ax.bar(x - 0.18, medias, 0.35, label="Media", color=PINK)
ax.bar(x + 0.18, medianas, 0.35, label="Mediana", color=CYAN)
ax.set_xticks(x)
ax.set_xticklabels(labels)
ax.legend()
ax.grid(axis="y", alpha=0.2)
plt.tight_layout()
plt.show()
guardar(fig, "fig7_media_vs_mediana.png")
```

Media vs Mediana (justifica usar mediana para imputar)

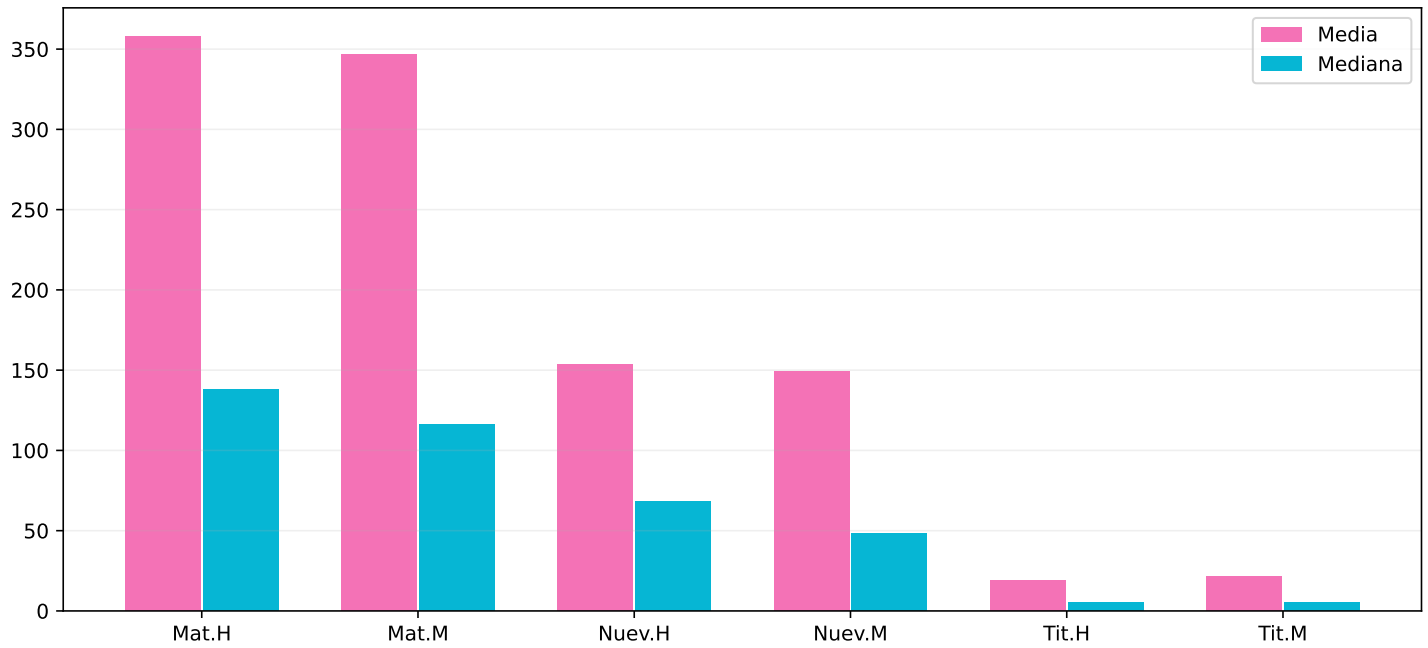


fig7_media_vs_mediana.png

```
fig, ax = plt.subplots(figsize=(12, 6))
fig.suptitle("Promedio: Pública vs Privada", fontsize=14, fontweight="bold")
tm = df_work.groupby("TIPO DE EDUCACIÓN")[COLS].mean()
ax.bar(x - 0.18, tm.loc["PUBLICA"], 0.35, label="Pública", color="CYAN")
ax.bar(x + 0.18, tm.loc["PRIVADA"], 0.35, label="Privada", color="ORANGE")
ax.set_xticks(x)
ax.set_xticklabels(labels)
ax.legend()
ax.grid(axis="y", alpha=0.2)
plt.tight_layout()
plt.show()
guardar(fig, "fig8_publica_vs_privada.png")
```

Promedio: Pública vs Privada

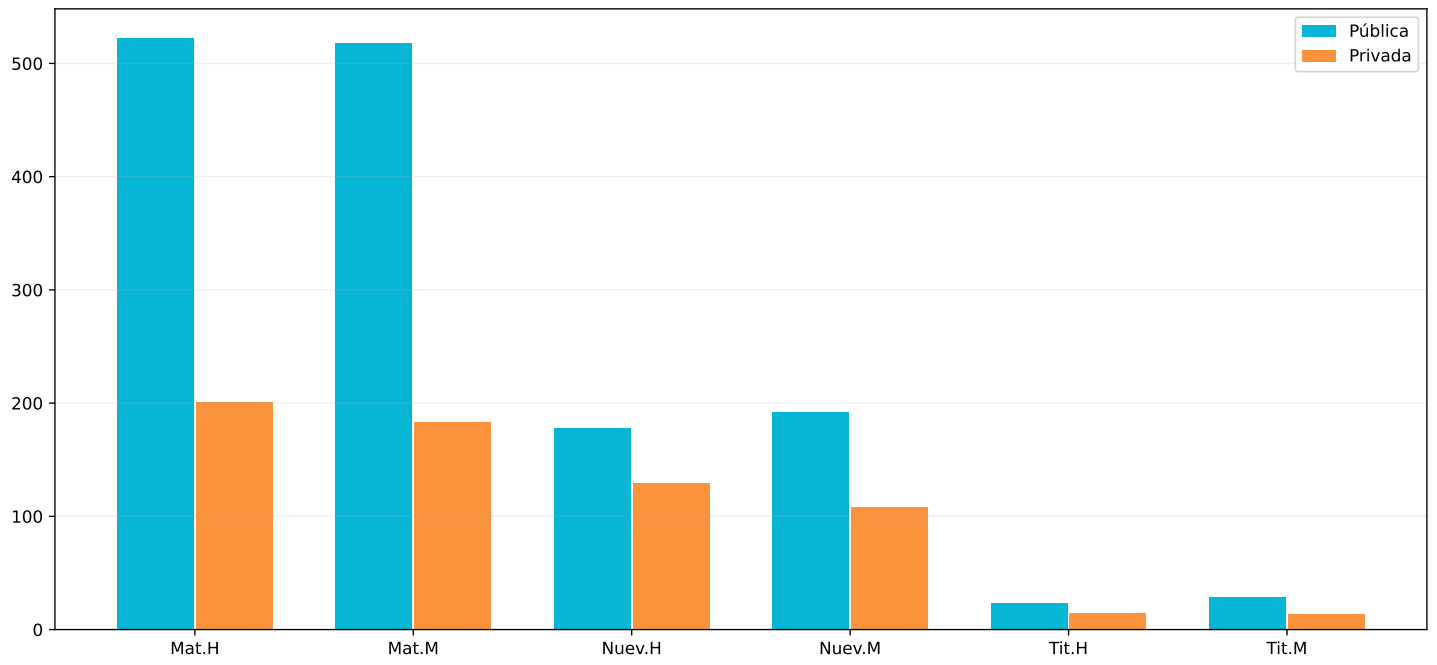


fig8_publica_vs_privada.png